RAG Scan Stack Management and Health

Contents

1 RAG Scan Stack - Management & Service Health Guide

Operations, Monitoring & Administrative Tasks

1.1 Table of Contents

1.2 1. Platform Administration Overview

1.2.1 Administrative Responsibilities

- [**TOOL**] **Service Health Monitoring**: Ensure all components are operational
- [**ANALYTICS**] **Performance Management**: Monitor resource usage and optimize performance

- ** Security Administration**: Manage access controls and audit logging
- ** Data Management**: Backup, recovery, and data lifecycle policies
- ** Update Management**: Apply security patches and feature updates
- [**GRAPH**] **Capacity Planning**: Scale resources based on testing demands

1.2.2 Admin Access Methods

```
# Dashboard Admin Interface
https://localhost:3002/settings  System Administration

# Direct Container Access
docker exec -it <container-name> /bin/bash

# Database Administration
docker exec -it rag-postgres psql -U app -d scans

# Log Monitoring
docker compose logs -f <service-name>
```

1.3 2. Service Health Monitoring

1.3.1 Health Dashboard Overview

1.3.1.1 Real-Time Service Status

Navigate to: **Dashboard Health Overview** or **Settings System Status**

```
Service Categories:
 Core Services: Essential platform components
 Scanner Services: Vulnerability scanning engines
```

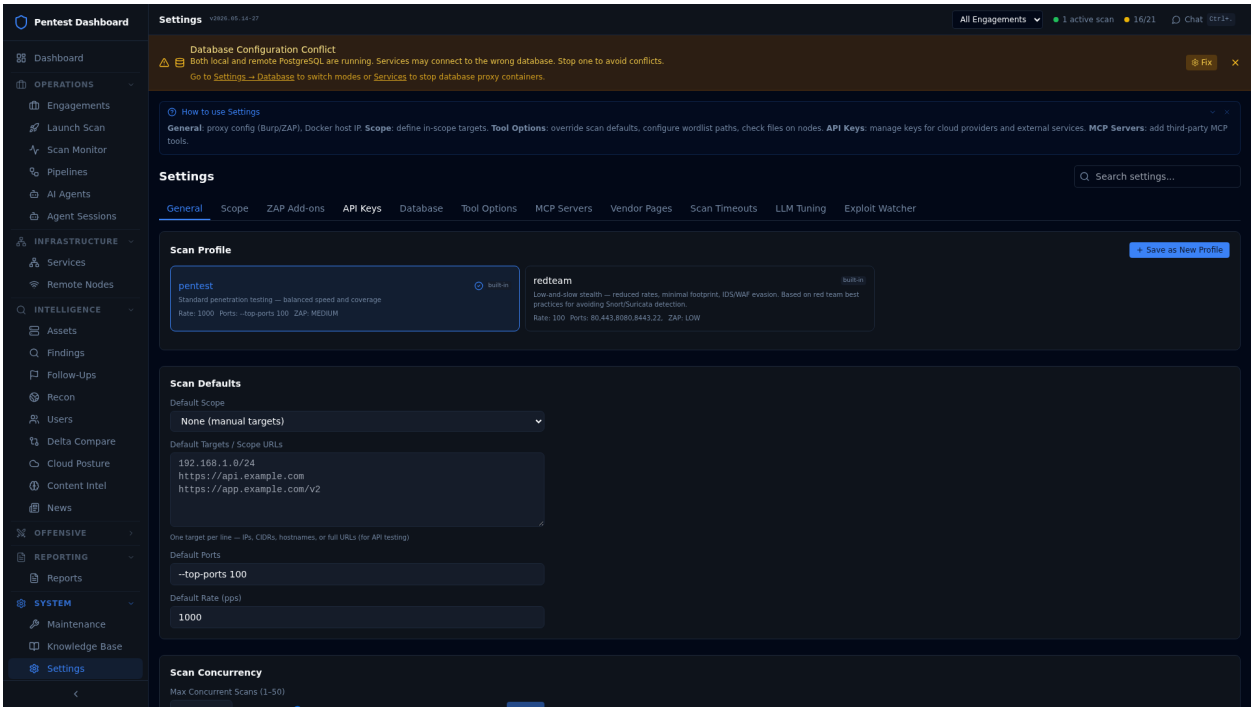

Figure 1: Health Dashboard

Figure 2: System Diagnostics

```
 AI Services: Machine learning and agent frameworks
 Optional Services: Enhanced features and integrations
 External Services: Third-party dependencies
```

1.3.1.2 Service Health Indicators

```
Status Colors:
[SUCCESS] Healthy (Green): Service operational, all checks passing
[WARNING] Degraded (Yellow): Service functional with non-critical issues
[ERROR] Unhealthy (Red): Service down or critical failure
 Stopped (Gray): Service intentionally disabled
 Starting (Blue): Service initializing or restarting
```

1.3.2 Core Service Monitoring

1.3.2.1 Essential Services (Must be Healthy)

```
[SUCCESS] rag-api: Main API server and data coordination
[SUCCESS] pentest-dashboard: Web interface and BFF proxy
[SUCCESS] rag-postgres: Primary database (if using local DB)
[SUCCESS] container-logs: Service orchestration and Docker management
[SUCCESS] embedder: Text processing and AI feature extraction
```

1.3.2.2 Critical Health Checks

- **Database Connectivity**: Connection pool status and query response times
- **API Response Times**: Service latency and throughput metrics
- **Memory Usage**: Container resource consumption and limits
- **Disk Space**: Storage utilization and available capacity
- **Network Connectivity**: Inter-service communication status

1.3.3 Scanner Service Monitoring

1.3.3.1 Vulnerability Scanners

```
[SEARCH] Scanning Services:
[SUCCESS] nmap_scanner: Port discovery and service enumeration
[SUCCESS] nuclei-runner: Template-based vulnerability detection
[SUCCESS] web-scanner: Web application security testing
[SUCCESS] playwright-scanner: Modern JavaScript application testing
[SUCCESS] osint-runner: Open source intelligence gathering
[SUCCESS] pd-runner: Network discovery and fingerprinting
[SUCCESS] brutus-runner: Credential testing and brute force
```

1.3.3.2 Scanner Health Metrics

- **Queue Depth**: Pending scan jobs and processing capacity
- **Tool Availability**: Scanner binary status and version checking
- **Template Updates**: Nuclei template refresh and signature currency
- **Resource Usage**: CPU and memory consumption during scans
- **Error Rates**: Failed scan percentages and common failure modes

---

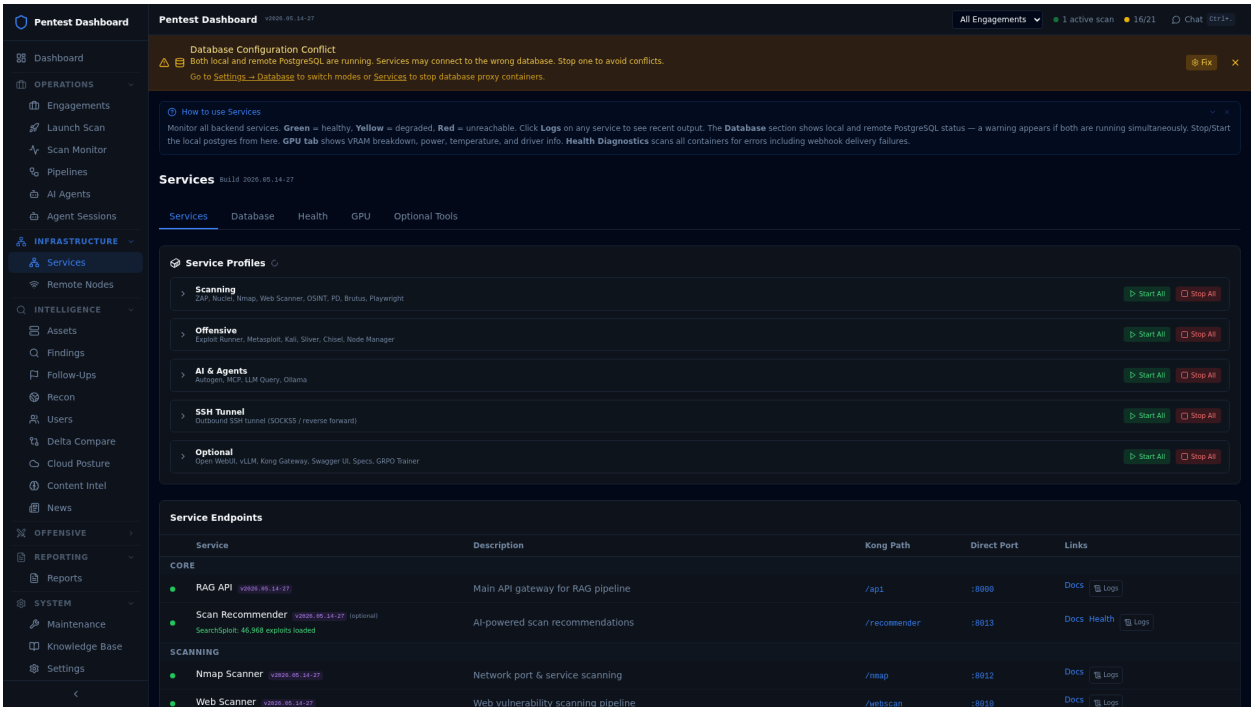

Figure 3: Services Management

Figure 4: Services Management

1.4 3. Container & Service Management

1.4.1 Service Control Interface

1.4.1.1 Navigate to: Settings Services

1.4.1.2 Profile-Based Service Groups

```
Service Profiles:
[TOOL] Core: Essential platform services (always running)
[SEARCH] Scan: Vulnerability scanning engines
 AI: Machine learning and agent services
[TARGET] Offensive: Red team and exploitation tools
 SSH-Tunnel: Remote access and tunneling
 Optional: Enhanced features (GPU, Kong, etc.)
```

1.4.1.3 Service Control Operations

```
Available Actions:
-> Start Profile: Launch all services in a profile group
 Stop Profile: Gracefully shutdown profile services
 Restart Profile: Stop and restart all services
[TOOL] Start Individual: Launch specific container
 Stop Individual: Stop specific container
[ANALYTICS] View Logs: Stream real-time container logs
[GRAPH] Resource Usage: Monitor CPU, memory, disk I/O
```

1.4.2 Container Management

1.4.2.1 Docker Compose Operations

```
# Profile Management
docker compose --profile gpu up -d        # Start GPU services
docker compose --profile optional up -d   # Start optional services
docker compose down                        # Stop all services

# Service-Specific Operations
docker compose restart rag-api           # Restart API server
docker compose logs -f nuclei-runner     # Stream scanner logs
docker compose ps                        # Show running containers
```

1.4.2.2 Individual Container Management

```
# Container Operations
docker stop <container-name>             # Graceful shutdown
docker start <container-name>            # Start stopped container
docker restart <container-name>          # Stop and start
docker exec -it <container-name> bash    # Interactive shell

# Resource Monitoring
docker stats                             # Real-time resource usage
docker inspect <container-name>          # Detailed container info
docker logs --tail 100 <container-name>  # Recent log output
```

1.4.3 Health Check Configuration

1.4.3.1 Automated Health Monitoring

```
Health Check Intervals:
- API Services: 15-20 second intervals
- Scanners: 20-30 second intervals
- Database: 10 second intervals
- Optional Services: 30 second intervals

Failure Thresholds:
- Retries: 3-5 attempts before marking unhealthy
- Timeout: 5-10 seconds per health check
- Start Period: 30-120 seconds for initialization
```

1.5 4. Performance Monitoring & Metrics

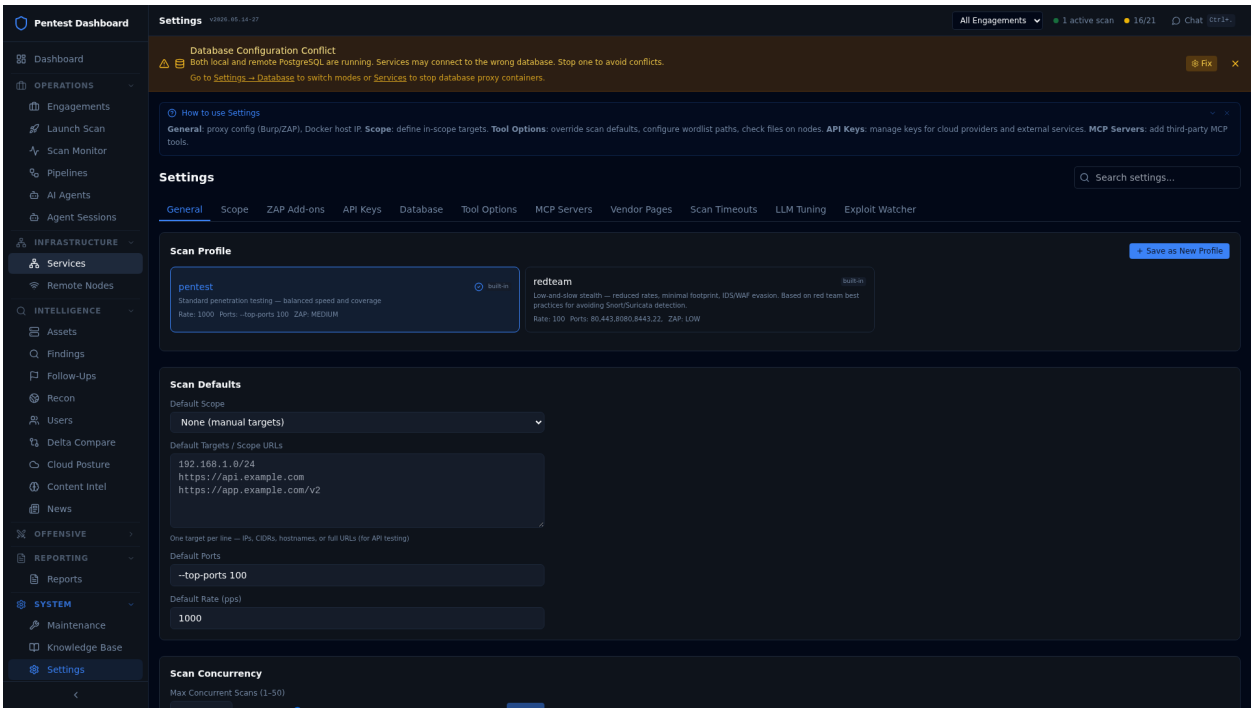

Figure 5: System Diagnostics

1.5.1 Resource Monitoring

1.5.1.1 System-Level Metrics

```
# Host Resource Usage
htop                    # Interactive process monitor
df -h                   # Disk space usage
free -h                 # Memory utilization
iostat                  # Disk I/O statistics
```

1.5.1.2 Container Resource Monitoring

- **CPU Usage**: Per-container processor utilization
- **Memory Consumption**: RAM usage and swap activity

Figure 6: OpSec Dashboard

- **Disk I/O**: Read/write operations and throughput
- **Network Traffic**: Inter-service communication volume
- **Storage Usage**: Persistent volume utilization

1.5.2 Performance Dashboards

1.5.2.1 Navigate to: Settings Diagnostics

1.5.2.2 Key Performance Indicators

```
Platform KPIs:
[ANALYTICS] Scan Throughput: Targets processed per hour
 Average Response Time: API latency percentiles
[TARGET] Success Rate: Successful scan completion percentage
 Data Processing: Findings ingested per minute
 Queue Performance: Job processing and wait times
```

1.5.2.3 Service-Specific Metrics

```
Per-Service Monitoring:
[SEARCH] Scanner Performance: Tool execution times and success rates
 AI Processing: Model inference latency and accuracy
 Database Performance: Query times and connection pooling
[NETWORK] Web Interface: Page load times and user interactions
```

1.5.3 Alerting & Notifications

1.5.3.1 Critical Alert Conditions

- **Service Downtime**: Core service failure or restart

- **Resource Exhaustion**: CPU, memory, or disk threshold exceeded
- **Database Issues**: Connection failures or slow queries
- **Security Events**: Authentication failures or unauthorized access
- **Scan Failures**: High error rates or tool unavailability

1.5.3.2 Alert Delivery Methods

- **Dashboard Notifications**: In-app alerts and status indicators
- **Email Alerts**: Administrative notifications for critical issues
- **Webhook Integration**: External monitoring system integration
- **Log-Based Alerts**: Pattern matching in service logs

1.6 5. Database Administration

1.6.1 Database Overview

1.6.1.1 Supported Configurations

- **Local PostgreSQL**: Single-node database in `rag-postgres` container
- **Remote PostgreSQL**: External database with SSH tunnel support
- **High Availability**: Master/replica configurations for production
- **Cloud Managed**: AWS RDS, Azure Database, GCP Cloud SQL integration

1.6.2 Database Health Monitoring

1.6.2.1 Navigate to: Settings System Status Database Section

1.6.2.2 Critical Database Metrics

```
Database KPIs:
 Active Connections: Current client connections vs. pool limits
[FAST] Query Performance: Average execution time and slow queries
 Storage Usage: Database size growth and available space
 Replication Status: Master/replica sync lag (if applicable)
[SECURE] Lock Contention: Blocking queries and deadlock frequency
```

1.6.2.3 Connection Pool Management

```
Pool Configuration:
- Max Connections: Typically 10-50 based on workload
- Idle Timeout: Automatic connection cleanup (300 seconds)
- Health Checks: Connection validation queries
- Overflow Handling: Queue management for peak demand
```

1.6.3 Database Maintenance

1.6.3.1 Regular Maintenance Tasks

```sql
-- Connection Status
SELECT count(*), state FROM pg_stat_activity GROUP BY state;

-- Database Size
SELECT pg_size_pretty(pg_database_size('scans'));

-- Active Queries
SELECT pid, usename, application_name, state, query
```

```
FROM pg_stat_activity WHERE state = 'active';

-- Index Usage
SELECT schemaname, tablename, indexname, idx_tup_read, idx_tup_fetch
FROM pg_stat_user_indexes ORDER BY idx_tup_read DESC LIMIT 10;
```

1.6.3.2 Backup & Recovery

```
# Database Backup
docker exec rag-postgres pg_dump -U app scans > backup.sql

# Full System Backup
docker run --rm -v rag-pgdata:/data -v $(pwd):/backup ubuntu \
  tar czf /backup/database-backup.tar.gz /data

# Recovery Operation
docker exec -i rag-postgres psql -U app scans < backup.sql
```

1.6.4 Schema Management

1.6.4.1 Migration Tracking

- **Version Control**: Database schema versioning with migration scripts
- **Rollback Capability**: Safe schema changes with rollback procedures
- **Testing**: Schema validation in development before production
- **Documentation**: Change logs and impact assessment

1.7 6. User Management & Security

1.7.1 Authentication & Authorization

1.7.1.1 User Access Control

```
Authentication Methods:
 API Key: Service-to-service authentication
 Session-Based: Web interface user sessions
 Certificate Auth: mTLS for service communication
 OAuth Integration: Enterprise SSO (planned feature)
```

1.7.1.2 Role-Based Access Control

```
User Roles:
 Administrator: Full system access and configuration
[TOOL] Operator: Scan execution and findings management
 Viewer: Read-only access to findings and reports
 Service Account: API-only access for integrations
```

1.7.2 Security Configuration

1.7.2.1 Navigate to: Settings Security

1.7.2.2 API Security Management

- **API Key Generation**: Secure key creation with expiration dates
- **Rate Limiting**: Request throttling to prevent abuse
- **IP Restrictions**: Source IP whitelist for sensitive operations

- **Audit Logging**: Complete access log with user attribution

1.7.2.3 Network Security

```
Security Controls:
[SECURE] TLS Encryption: All service communication encrypted
 Container Isolation: Network segmentation between services
 Firewall Rules: Minimal exposed ports and access control
 Certificate Management: Automated SSL certificate rotation
```

1.7.3 Audit & Compliance

1.7.3.1 Security Monitoring

- **Access Logs**: User authentication and authorization events
- **Configuration Changes**: System modification tracking with rollback
- **Data Access**: Findings and sensitive data access logging
- **Failed Attempts**: Security event monitoring and alerting

1.7.3.2 Compliance Features

- **Data Retention**: Configurable retention policies for findings
- **Export Controls**: Secure data export with audit trails

- **Privacy Controls**: PII handling and data anonymization
- **Regulatory Reporting**: Compliance report generation

1.8 7. Backup & Recovery

1.8.1 Backup Strategy

1.8.1.1 Automated Backup Components

```
Backup Scope:
 Database: Full PostgreSQL backup with point-in-time recovery
 Configuration: Service configurations and environment settings
 Secrets: Encrypted backup of keys, certificates, and credentials
[ANALYTICS] Scan Data: Findings, reports, and evidence files
[GRAPH] Metrics: Historical performance and audit data
```

1.8.1.2 Backup Frequency & Retention

```
Backup Schedule:
 Daily: Full database backup with 30-day retention
 Hourly: Transaction log backup for point-in-time recovery
 Weekly: Complete system backup with 90-day retention
 Monthly: Archive backup with 1-year retention
```

1.8.2 Recovery Procedures

1.8.2.1 Recovery Scenarios

1. **Service Restart**: Container or service-level recovery
2. **Data Corruption**: Database rollback to known good state
3. **Complete System Failure**: Full platform restoration
4. **Partial Data Loss**: Selective recovery of specific components

1.8.2.2 Recovery Time Objectives

```
Recovery Metrics:
[TARGET] RTO (Recovery Time Objective): 2-4 hours for full system
[ANALYTICS] RPO (Recovery Point Objective): 1 hour maximum data loss
[FAST] Service Restart: 5-15 minutes for individual services
 Database Recovery: 30-60 minutes for full restoration
```

1.8.3 Disaster Recovery

1.8.3.1 High Availability Configuration

- **Database Replication**: Master/standby configuration with automatic failover
- **Load Balancing**: Distributed service deployment across multiple nodes

- **Geographic Distribution**: Multi-site deployment for disaster recovery
- **Cloud Integration**: Hybrid on-premise/cloud backup strategies

---

1.9 8. Troubleshooting Guide

1.9.1 Common Issues & Solutions

1.9.1.1 Service Startup Failures

```
# Check container status
docker compose ps

# View startup logs
docker compose logs <service-name>

# Common solutions
docker compose pull                    # Update images
docker compose down && docker compose up -d  # Full restart
docker system prune                    # Clean up resources
```

1.9.1.2 Database Connection Issues

```
# Test database connectivity
docker exec rag-postgres pg_isready -U app -d scans

# Check connection pool
docker exec rag-api python -c "
from api import db
print(db.engine.pool.status())
"

# Reset connections
docker compose restart rag-api rag-postgres
```

1.9.1.3 Scanner Tool Failures

```
# Check tool availability
docker exec nuclei-runner nuclei -version
docker exec nmap_scanner nmap --version

# Update scanner databases
```

```
docker exec nuclei-runner nuclei -update-templates
docker exec osint-runner /usr/bin/update-tools.sh

# Clear scan queue
docker exec rag-api python -c "
from api import clear_scan_queue
clear_scan_queue()
"
```

1.9.2 Performance Issues

1.9.2.1 High Resource Usage

```
# Identify resource-hungry containers
docker stats --no-stream

# Check system resources
df -h                           # Disk space
free -h                         # Memory usage
top -p $(pgrep -d',' -f docker) # Docker processes
```

1.9.2.2 Slow Response Times

1. **Database Performance**: Check slow queries and connection pool
2. **Network Latency**: Verify inter-service communication
3. **Resource Contention**: Monitor CPU and memory utilization
4. **Disk I/O**: Check storage performance and available space

1.9.3 Log Analysis

1.9.3.1 Centralized Log Collection

```
# View all service logs
docker compose logs --tail=100 -f

# Service-specific logs
docker compose logs rag-api --tail=50
docker compose logs nuclei-runner --since=1h

# Error pattern search
docker compose logs | grep -i error
docker compose logs | grep -i "failed\|exception\|timeout"
```

1.9.3.2 Log Level Configuration

- **DEBUG**: Detailed execution information for development
- **INFO**: Normal operation events and status updates
- **WARN**: Non-critical issues that may need attention
- **ERROR**: Service errors requiring immediate investigation
- **CRITICAL**: System failures requiring immediate response

1.10 9. Maintenance Procedures

1.10.1 Regular Maintenance Tasks

1.10.1.1 Daily Maintenance

```
# Health check verification
curl -k https://localhost:3002/api/health

# Database maintenance
docker exec rag-postgres psql -U app -d scans -c "VACUUM ANALYZE;"

# Log rotation
docker system prune --filter "until=24h"

# Resource monitoring
docker stats --no-stream | head -10
```

1.10.1.2 Weekly Maintenance

```
# Update scanner templates
docker exec nuclei-runner nuclei -update-templates
docker exec osint-runner /scripts/update-wordlists.sh

# Database optimization
docker exec rag-postgres psql -U app -d scans -c "REINDEX DATABASE scans;"

# Security scanning
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy image --severity HIGH,CRITICAL rag-scan-stack-*
```

1.10.1.3 Monthly Maintenance

- **System Updates**: Apply security patches and minor updates
- **Capacity Planning**: Review resource usage trends and plan scaling
- **Security Audit**: Review access logs and security configurations
- **Backup Verification**: Test backup restore procedures
- **Performance Review**: Analyze metrics and optimize configurations

1.10.2 Update Management

1.10.2.1 Version Control Strategy

```
Update Categories:
[SECURE] Security Patches: Apply immediately for critical vulnerabilities
 Bug Fixes: Apply during scheduled maintenance windows
 Feature Updates: Plan deployment with stakeholder approval
[TOOL] Configuration Changes: Test in development before production
```

1.10.2.2 Update Procedure

1. **Backup Current State**: Full system backup before changes
2. **Update in Development**: Test updates in non-production environment
3. **Plan Maintenance Window**: Schedule downtime with stakeholders
4. **Apply Updates**: Execute changes with rollback capability
5. **Validate Functionality**: Comprehensive testing post-update
6. **Document Changes**: Update documentation and notify users

1.10.3 Monitoring & Alerting Maintenance

1.10.3.1 Alert Tuning

- **Threshold Adjustment**: Optimize alert thresholds based on historical data

- **False Positive Reduction**: Refine alert conditions to reduce noise
- **Escalation Procedures**: Update contact information and escalation paths
- **Integration Testing**: Verify external monitoring system integration

---

1.11 Emergency Procedures

1.11.1 Critical System Failure

1. **Immediate Response**: Assess scope of failure and potential causes
2. **Service Isolation**: Stop affected services to prevent cascade failures
3. **Stakeholder Notification**: Alert management and affected users
4. **Recovery Execution**: Follow documented recovery procedures
5. **Root Cause Analysis**: Post-incident investigation and remediation
6. **Process Improvement**: Update procedures based on lessons learned

1.11.2 Security Incident Response

1. **Incident Detection**: Identify and classify security events
2. **Containment**: Isolate affected systems and limit damage
3. **Evidence Preservation**: Secure logs and forensic evidence
4. **Communication**: Notify security team and management
5. **Recovery**: Restore systems to secure operational state
6. **Post-Incident Review**: Analyze response and improve security controls

---

1.12 Conclusion

Effective management of the RAG Scan Stack requires proactive monitoring, regular maintenance, and rapid response to issues. This guide provides the foundation for maintaining a healthy, secure, and high-performing security testing platform.

Key Success Factors: - [SUCCESS] **Proactive Monitoring**: Continuous health checks and performance monitoring - [SUCCESS] **Automated Maintenance**: Scheduled tasks and automated recovery procedures - [SUCCESS] **Documentation**: Maintain current procedures and configuration documentation - [SUCCESS] **Training**: Ensure operational team understands platform architecture and procedures - [SUCCESS] **Testing**: Regular disaster recovery and incident response testing

For Additional Support: - ** Technical Documentation**: Platform architecture and API reference** - Troubleshooting Wiki**: Community-contributed solutions and best practices** - Emergency Contacts**: Escalation procedures for critical issues** - Operations Channel**: Team communication for coordination and knowledge sharing